Chapter 6: SQA tools

Reference

This chapter refers from the book:

Mastering Software Quality Assurance: Best Practices, Tools and Techniques for Software Developers

Introduction to Software Testing

Outline

6.1. QA management tools

6.2. Unit testing tool

6.3. Automated functional testing tool

6.4. Performance testing tool

6.5. Security testing tool

6.6. Applying SQA tools in projects

Outline

6.1. QA management tools

6.2. Unit testing tool

6.3. Automated functional testing tool

6.4. Performance testing tool

6.5. Security testing tool

6.6. Applying SQA tools in projects

6.1. QA management tools

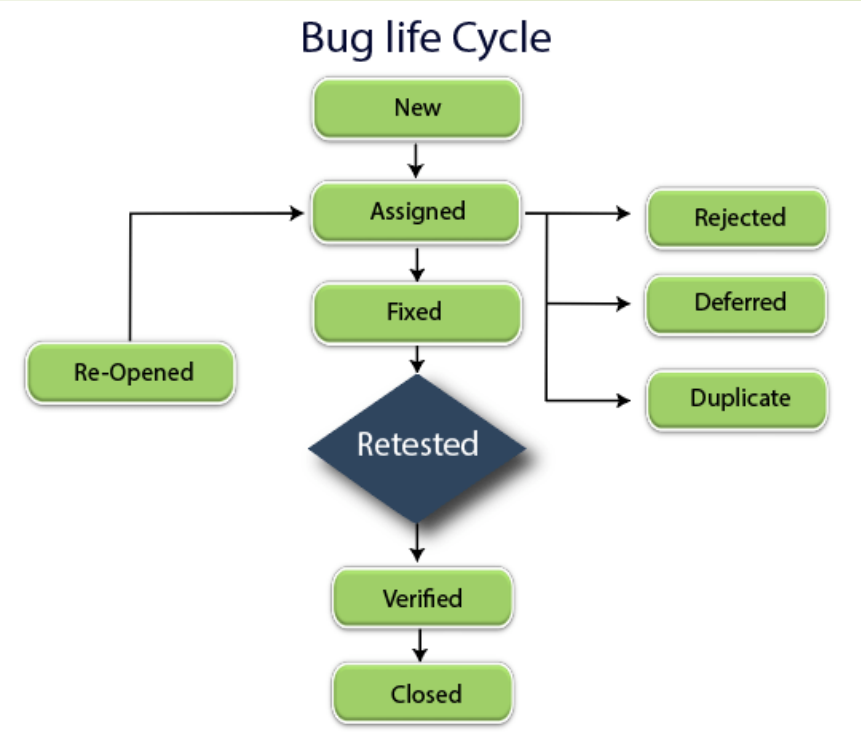
- QA management tools assist testers in doing their tasks effectively. Their tasks including:
  1. Creating and maintaining release/project cycle/component information.
  2. Creating and maintaining the test artifacts specific to each release/cycle for which we have- requirements, test cases, etc.
  3. Establishing traceability and coverage of the test assets.
  4. Test execution support – test suite creation, test execution status capture, etc.
  5. Metric collection/report-graph generation for analysis.
  6. Bug tracking/defect management.

Documentation support software

- Writing reports, user manuals, designing and building test cases, and other documentation-related tasks frequently employ documentation support software.
- The most commonly used software are:
- Microsoft Office Word.
- Microsoft Office Exel.
- Microsoft Office Project.
- Microsoft Office Power Point.

Bug tracking system

- A bug tracking system is required for software projects in order to track and report defects while the software is being developed.



Bug tracking system (cont.)

- Following are some popular bug tracking softwares:
  - ***Bugzilla:*** http://www.bugzilla.org/
  - ***Redmine:*** http://www.redmine.org/
  - ***Atlassian JIRA :*** http://www.atlassian.com

Bug tracking system: Bugzilla

- **Bugzilla** is server software designed to help you manage software development.
- Key features of Bugzilla includes
  - Advanced search capabilities
  - E-mail Notifications
  - Modify/file Bugs by e-mail
  - Time tracking
  - Strong security
  - Customization
  - Localization

Bug tracking system: Redmine

- Redmine is a flexible project management web application written using Ruby on Rails framework.
- Some of the main features of Redmine are:
  - Multiple projects support
  - Flexible role based access control
  - Flexible issue tracking system
  - Gantt chart and calendar
  - News, documents & files management
  - Feeds & email notifications
  - Per project wiki
  - Per project forums
  - Time tracking
  - Custom fields for issues, time-entries, projects and users
  - SCM integration (SVN, CVS, Git, Mercurial and Bazaar)
  - Issue creation via email
  - …

Bug tracking system: Atlassian JIRA

- Built for every member of agile teams and beyond to plan, track, and ship world-class software.
- Use cases
  - Agile teams
  - Bug tracking
  - Project management
  - Product management
  - Process management
  - Task management
  - Software development
  - Requirements & test case management

Jira Software

Outline

6.1. QA management tools

6.2. Unit testing tool

6.3. Automated functional testing tool

6.4. Performance testing tool

6.5. Security testing tool

6.6. Applying SQA tools in projects

6.2. Unit testing tool

- Unit testing tools are used by the developers to test the source code of the application .

JUNIT5 introduction

- JUnit is an open source Unit Testing Framework for JAVA. It is useful for Java Developers to write and run repeatable tests.

A first test case

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import example.util.Calculator;
import org.junit.jupiter.api.Test;
class MyFirstJUnitJupiterTests {
        private final Calculator calculator = new Calculator();
        @Test
        void addition() {
                assertEquals(2, calculator.add(1, 1));
        }
}
```

From: http://junit.org/junit5/docs/current/user-guide/

Junit Anotation

- JUnit Jupiter uses annotations for configuring tests and extending the framework.
- Examples:

Annotation	Description
@Test	Denotes that a method is a test method. Unlike JUnit 4's @Test annotation, this annotation does not declare any attributes, since test extensions in JUnit Jupiter operate based on their own dedicated annotations. Such methods are *inherited* unless they are *overridden*.
@BeforeEach	Denotes that the annotated method should be executed *before* **each** @Test, @RepeatedTest, @ParameterizedTest, or @TestFactory method in the current class; analogous to JUnit 4's @Before. Such methods are *inherited* – unless they are *overridden* or *superseded* (i.e., replaced based on signature only, irrespective of Java's visibility rules).
@AfterEach	Denotes that the annotated method should be executed *after* **each** @Test, @RepeatedTest, @ParameterizedTest, or @TestFactory method in the current class; analogous to JUnit 4's @After. Such methods are *inherited* – unless they are *overridden* or *superseded* (i.e., replaced based on signature only, irrespective of Java's visibility rules).

A standard test class

```
import static
org.junit.jupiter.api.Assertions.f
ail;
import static
org.junit.jupiter.api.Assumptions
.assumeTrue;
import
org.junit.jupiter.api.AfterAll;
import
org.junit.jupiter.api.AfterEach;
import
org.junit.jupiter.api.BeforeAll;
import
org.junit.jupiter.api.BeforeEach;
import
org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.Test;
class StandardTests {
        @BeforeAll
```

```
        @Test
        @Disabled("for
demonstration purposes")
        void skippedTest() { //
not executed }
        @Test
        void abortedTest() {

        assumeTrue("abc".con
ains("Z"));
                fail("test should
have been aborted");
        }
        @AfterEach
        void tearDown() { }
        @AfterAll
        static void
tearDownAll() { }
```

Class Assertions

Assertions is a collection of utility methods that support asserting conditions in tests.

Modifier and Type	Method	Description
static void	**assertEquals**(byte expected, byte actual)	*Assert* that expected and actual are equal.
static void	**assertNotEquals**(byte unexpected, byte actual)	*Assert* that expected and actual are not equal.
static void	**assertFalse**(boolean condition)	*Assert* that the supplied condition is false.
static void	**assertTrue**(boolean condition)	*Assert* that the supplied condition is true.

Outline

6.1. QA management tools

6.2. Unit testing tool

6.3. Automated functional testing tool

6.4. Performance testing tool

6.5. Security testing tool

6.6. Applying SQA tools in projects

6.3. Automated functional testing tool

- There are many automated functional testing tools, both free and paid, for example:
  - Selenium free tool, website testing
  - Unified Functional Testing (UFT) a paid tool, testing websites and software running on computers
  - Appium free tool, mobile app testing

- Among the above tools, Selenium is widely used due to its free powerful tool and strong support community.

Selenium introduction

- Selenium (https://www.selenium.dev) is an open-source and a portable automated software testing tool for testing web applications.
- Selenium is a set of tools that helps testers to automate web-based applications more efficiently:
  - **Selenium IDE:** Selenium Integrated Development Environment (IDE) A Firefox plugin enables testers to record their actions as they go through the process they need to test.
  - **Selenium RC:** Selenium Remote Control (RC) The pioneering testing framework that allowed for more than just straightforward browser activities and linear execution.
  - **Selenium WebDriver:** Selenium WebDriver is the replacement for Selenium RC, which delivers commands to the browser directly and returns results..
  - **Selenium Grid:** Selenium Grid is is a technology that reduces execution time by running parallel tests across multiple computers and browsers at once.

Outline

6.1. QA management tools

6.2. Unit testing tool

6.3. Automated functional testing tool

6.4. Performance testing tool

6.5. Security testing tool

6.6. Applying SQA tools in projects

6.4. Performance testing tool

- Performance testing is excuted to find out how quickly a system can handle a specific workload. It can also be used to confirm and validate the system's other high-quality characteristics, such as scalability, dependability, and resource efficiency.

Some performance testing tools

- **Loadrunner:** is a load/stress testing tool for websites and other applications, supporting a wide range of application environments, platforms, and databases. Link to homepage: https://www.microfocus.com/en-us/products/loadrunner-enterprise/overview
- **Apache JMeter:** can be used for performance testing of both static and dynamic resources. Link to homepage: http://jmeter.apache.org/

Performance testing tool: Loadrunner

- Support performance testing for the widest range of protocols and more than 50 technologies and application environments
- Quickly identify the most likely causes of performance issues with a patented auto-correlation engine.
- Accurately predict application scalability and capacity with accurate emulation of realistic loads.
- Flexible Test Scenarios: Run high-scale tests with minimal hardware and leverage the public cloud to scale up or down.
- Scripting: Easily create, record, correlate, replay, and enhance scripts for better load testing.
- Continuous Testing: Built-in integrations include IDE, CI/CD, open source test automation, monitoring, and source code management tools.

Performance testing tool: Apache JMeter:

- Apache JMeter features include:
  - Ability to load and performance test many different applications/server/protocol types.
  - Web - HTTP, HTTPS (Java, NodeJS, PHP, ASP.NET, …)
  - Full featured Test IDE that allows fast Test Plan recording (from Browsers or native applications), building and debugging.
  - CLI mode (Command-line mode (previously called Non GUI) / headless mode) to load test from any Java compatible OS (Linux, Windows, Mac OSX, …)
  - A complete and ready to present dynamic HTML report
  - Easy correlation through ability to extract data from most popular response formats, HTML, JSON , XML or any textual format
  - Complete portability and 100% Java purity.
  - Full multi-threading framework allows concurrent sampling by many threads and simultaneous sampling of different functions by separate thread groups.
  - Caching and offline analysis/replaying of test results.
  - Highly Extensible core:

Outline

6.1. QA management tools

6.2. Unit testing tool

6.3. Automated functional testing tool

6.4. Performance testing tool

6.5. Security testing tool

6.6. Applying SQA tools in projects

6.5. Security testing tool

- We utilize security testing to ensure that data in some information systems remains secure and cannot be accessed by unauthorized individuals. Software that have undergone successful security testing are shielded against harmful malware and other dangers that could cause them to crash or behave in an unexpected way.
- To check the vulnerabilities and defects in the software, there are numerous free, commercial, and open-source solutions accessible. Aside from being free, the nicest thing about open-source technologies is that we can modify them to meet your unique needs.

Some security testing tools

- **Zed Attack Proxy (ZAP)**: is a multi-platform, open-source online application security testing tool created by OWASP (Open Web Application Security Project). ZAP is used to identify a variety of security flaws in a web application both during the development and testing phases.
- **Wapiti**: Wapiti is an open source project from SourceForge and devloop that is free to use. In order to check web applications for security vulnerabilities, Wapiti employs black box testing. It's a command-line application,
- **W3af**: W3af is one of the most well-known frameworks for web application security testing that was created using Python. The program enables testers to identify more than 200 different kinds of security flaws in web applications.

Outline

6.1. QA management tools

6.2. Unit testing tool

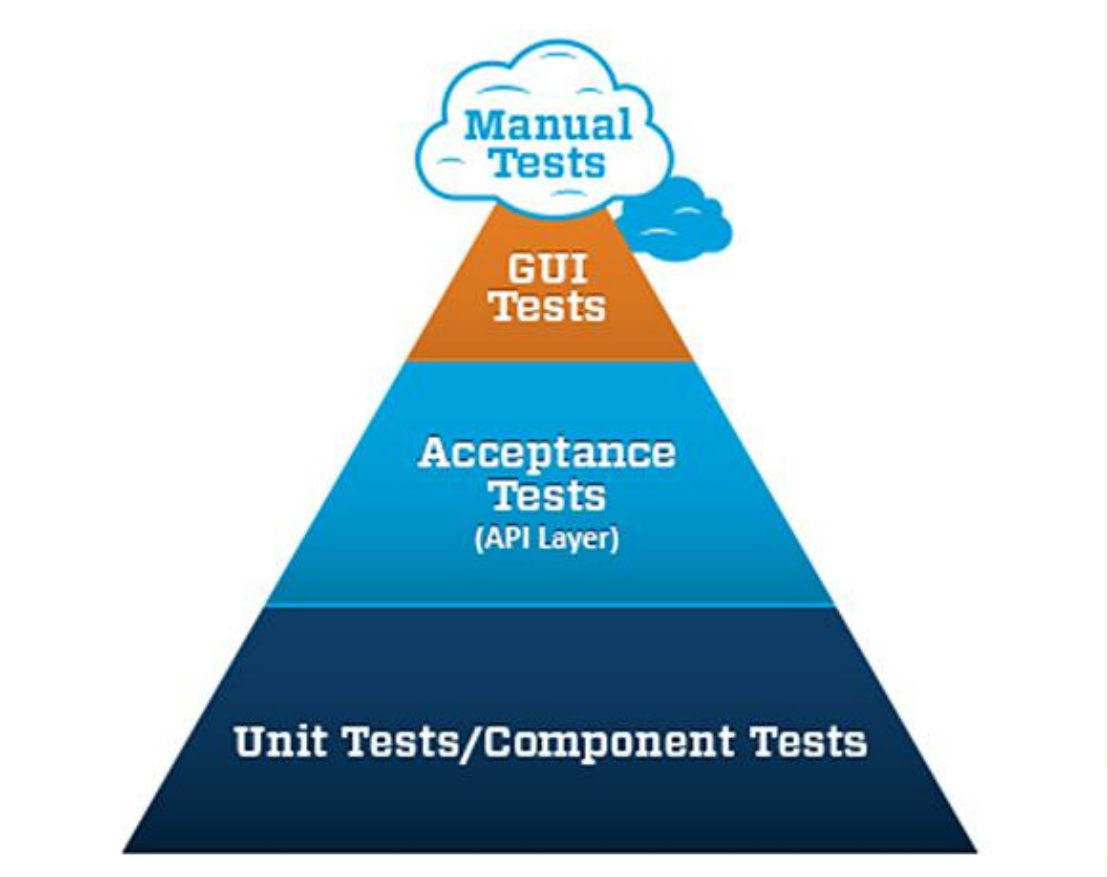
6.3. Automated functional testing tool

6.4. Performance testing tool

6.5. Security testing tool

6.6. Applying SQA tools in projects

6.6. Applying QA tools in projects



Software testing tools

Type	Examples
Functional test automation	Se, HP UFT
Mobile test automation	appium
Performance test	APACHE JMeter™
Test management	Jira
Security test	w3af Web Application Attack and Audit Framework, OWASP Zed Attack Proxy
API test	POSTMAN